SPRINT PLANNING DOCUMENT

File Compression Tool – Basic ZIP Implementation

Team: 05
Technology: C / C++
Algorithm: Huffman Coding

1. Project Overview

The File Compression Tool is a C/C++ application that uses Huffman Coding to compress files and decompress them back to their original form.

The main objective is to develop a lossless file compression system that reduces file size while ensuring that the decompressed output is identical to the original file.

2. Sprint 1 – Core Compression Implementation

Sprint Duration: October 19 – October 23, 2026

Sprint Goal: Develop the fundamental components required for file compression.

Planned Tasks and Activities

Issue	Task	Description
#1	File Input & Binary Reader	Read input files in binary mode and handle file input operations.
#2	Frequency Analysis Module	Calculate the frequency of each byte in the input file.
#3	Huffman Tree & Prefix Codes	Construct the Huffman tree and generate prefix-free binary codes.
#4	Encoder & Header Generator	Encode input data into a compressed bitstream and generate the required file header and metadata.

Sprint 1 Activities

- Implement file reading and binary input handling.
- Develop byte-frequency calculation.
- Construct the Huffman tree using frequency information.
- Generate Huffman prefix codes.
- Implement encoding and compressed-file header generation.
- Perform relevant unit and integration testing.

Expected Deliverables

- File input and binary reader module.
- Frequency analysis module.
- Huffman tree construction.
- Prefix-code generation.
- Encoder and header generation functionality.

3. Sprint 2 – Decompression, Validation and Integration

Sprint Duration: October 26 – October 30, 2026

Sprint Goal: Develop decompression functionality, validate compressed files, reconstruct original data and integrate the command-line workflow.

Planned Tasks and Activities

Issue	Task	Description
#5	Decompression Input & Validator	Accept compressed files and validate input before decompression.
#6	Huffman Decoder & Reconstruction	Reconstruct the Huffman decoding tree and decode the compressed bitstream.
#7	Output Writer & Integrity Check	Write the decompressed output and verify data integrity.
#8	CLI, Statistics & Error Handling	Handle command-line operations, display compression statistics and manage errors.

Sprint 2 Activities

- Implement compressed-file input validation.
- Reconstruct the Huffman decoding tree.
- Decode the compressed bitstream.
- Recover the original byte stream.
- Write decompressed data to the output file.
- Verify the integrity of decompressed files.
- Implement CLI command handling and parsing.
- Display relevant compression statistics.
- Handle invalid inputs and errors.
- Perform integration and regression testing.

Expected Deliverables

- Decompression input validator.
- Huffman decoder and tree reconstruction.
- Output writer and integrity checking.
- CLI command handling and parser.
- Compression statistics and error handling.
- Integrated compression and decompression workflow.

5. Testing Strategy

Testing is performed throughout both sprints.

Sprint 1 Testing

- Verify file input and binary reading.
- Check byte-frequency calculations.
- Verify Huffman tree construction.
- Check prefix-free code generation.
- Test encoding and compressed-file header generation.

Sprint 2 Testing

- Test valid compressed-file input.
- Test invalid or corrupted headers.
- Verify Huffman tree reconstruction.
- Test compressed bitstream decoding.
- Check decompressed output integrity.
- Test invalid compressed bit counts.
- Verify CLI commands and error handling.
- Perform integration and regression testing.